

# Platform Amendment — Version 4.9.0

## Vermont Health Platform (HTR)

**Type:** Amendment to Platform Documentation (supplements v4.8.0 and all prior amendments) **Version:** 4.9.0  
**Date:** April 2026 **Classification:** Internal **Scope:** Vercel deployment sprint — complete record of the monorepo build configuration, dependency resolution issues, TypeScript build errors, security header configuration, and the working solution that emerged from extended back-and-forth. Supplements the first-deployment walkthrough in [docs/deployment-guide.md](#) (v4.2.0), which covers service setup but does not address the structural complexity of deploying a Next.js app from a monorepo subdirectory.

### Table of Contents

- 1. [Why Vercel Deployment Is Non-Trivial for This Repo](#)
- 2. [Repository Structure](#)
- 3. [The Working Vercel Configuration](#)
- 4. [Dependency Resolution — The Legacy Peer Deps Problem](#)
- 5. [Approaches Tried and Abandoned](#)
- 6. [TypeScript Build Errors Fixed for Vercel](#)
- 7. [Suspense Boundary Requirement — useSearchParams](#)
- 8. [Security Headers — next.config.ts](#)
- 9. [Sentry Integration](#)
- 10. [Environment Variables — Complete Reference](#)
- 11. [Cron Job — Nightly Revalidation](#)
- 12. [CORS Headers](#)
- 13. [Relationship to Existing deployment-guide.md](#)
- 14. [Deployment Checklist](#)

### 1. Why Vercel Deployment Is Non-Trivial for This Repo

Most Vercel tutorials assume the repository root IS the Next.js application. This repo does not follow that pattern.

The codebase is structured as a **monorepo** with the Next.js frontend living in a **frontend/** subdirectory. Python backend code lives separately under **backend/**. The repo root itself is not a Node.js package.

This creates three problems Vercel does not handle automatically:

- 1. **Install location** — `npm install` at the root installs nothing useful unless a `package.json` exists there (and the packages are declared in it). Vercel's default assumes the root is the app root.
- 2. **Build path** — Vercel needs to know to enter **frontend/** and run `next build` there — but `next` must be resolvable, which requires the install to have happened first.
- 3. **Output directory** — Vercel looks for the build artifacts in `.next/` at the root by default. The actual output is at `frontend/.next/`.

All three require explicit override in `vercel.json`.

### 2. Repository Structure

|                          |                                            |
|--------------------------|--------------------------------------------|
| Vermont-Health-Platform/ | ← git root / Vercel project root           |
|                          |                                            |
| ├─ vercel.json           | ← ACTIVE Vercel config (root-level)        |
| ├─ .npmrc                | ← npm config (legacy-peer-deps=true)       |
| ├─ package.json          | ← root package (lists all deps for Vercel) |
| ├─ package-lock.json     | ← root lockfile                            |
|                          |                                            |



**Key point:** `vercel.json` sits at the repo root and explicitly tells Vercel where to install, where to build, and where to find output. Without it, Vercel would try to treat the root as the app and fail immediately.

### 3. The Working Vercel Configuration

**File:** `vercel.json` (repo root)

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "installCommand": "npm install",
  "buildCommand": "cd frontend && ../node_modules/.bin/next build",
  "outputDirectory": "frontend/.next",
  "framework": "nextjs",
  "regions": ["iad1"],

  "headers": [
    {
      "source": "/api/(.*)",
      "headers": [
        { "key": "Access-Control-Allow-Methods", "value": "GET, POST, OPTIONS" },
        { "key": "Access-Control-Allow-Headers", "value": "Content-Type, Authorization" }
      ]
    }
  ],

  "crons": [
    {
      "path": "/api/cron/revalidate",
      "schedule": "0 0 * * *"
    }
  ]
}
```

Why each field exists

| Field                       | Value                                                               | Why                                                                                                                       |
|-----------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>installCommand</code> | <code>npm install</code>                                            | Runs at repo root, installs into <code>node_modules/</code> at root                                                       |
| <code>buildCommand</code>   | <code>cd frontend &amp;&amp; ../node_modules/.bin/next build</code> | Enters <code>frontend/</code> , invokes <code>next</code> binary from root <code>node_modules</code> (not a PATH command) |

| Field           | Value          | Why                                                                                |
|-----------------|----------------|------------------------------------------------------------------------------------|
| outputDirectory | frontend/.next | Tells Vercel where to find the built artifacts                                     |
| framework       | nextjs         | Enables Next.js-specific Vercel features (ISR, edge functions, image optimization) |
| regions         | ["iad1"]       | US East (Northern Virginia) — closest datacenter to Vermont users                  |

Why `../node_modules/.bin/next build` and not `npx next build` or just `next build`

After `npm install` runs at the repo root, the `next` binary is at `node_modules/.bin/next` relative to the root. Inside the `buildCommand`, we `cd frontend` first — which means the working directory is now `frontend/`, and `../node_modules/.bin/next` correctly resolves back to the root's `node_modules`.

`npx next build` would attempt to download Next.js from npm instead of using the installed version — unreliable in CI. A bare `next build` would fail because `next` is not on PATH in Vercel's build environment after a root install.

## 4. Dependency Resolution — The Legacy Peer Deps Problem

The conflict

`@sentry/nextjs` — the Sentry SDK for Next.js — declares a peer dependency that requires a specific range of Next.js versions. When Next.js 16 was in use, `@sentry/nextjs` had not yet published a version with a compatible peer dependency declaration. Running `npm install` with strict peer resolution (npm v7+ default) would error:

```
npm error code ERESOLVE
npm error ERESOLVE unable to resolve dependency tree
npm error
npm error While resolving: ...
npm error Found: next@16.x.x
npm error node_modules/next
npm error   next@"^16.0.0" from the root project
npm error
npm error Could not resolve dependency:
npm error   peer next@">=13.2.0 <16" from @sentry/nextjs@...
```

Chronology of fixes tried

| Attempt | What                                                                              | Result                                                      |
|---------|-----------------------------------------------------------------------------------|-------------------------------------------------------------|
| 1       | Downgrade Next.js to 15                                                           | Resolved the peer dep — but Next.js 16 features were needed |
| 2       | Restore Next.js 16, add <code>legacy-peer-deps=true</code> to <code>.npmrc</code> | ✅ Works — Sentry installs, app builds                       |

The solution

File: `.npmrc` (repo root)

```
legacy-peer-deps=true
```

This single line instructs npm to use npm v6-style peer dependency resolution, which installs packages even when peer dep ranges are not perfectly satisfied, instead of erroring. It is a **project-wide setting** because Vercel reads `.npmrc` from the repo root before running `installCommand`.

**Important:** This flag should remain in `.npmrc` until `@sentry/nextjs` releases a version with explicit Next.js 16 peer dep support. Removing it will cause Vercel builds to fail again.

---

## 5. Approaches Tried and Abandoned

The following were attempted during the deployment sprint and ultimately reverted. They are documented here so future developers do not repeat the same detours.

### Approach A — Symlink node\_modules to repo root

**Commit:** `fix: symlink node_modules to repo root for Vercel module resolution`

The idea was to install in `frontend/` (where Vercel normally expects) and then symlink `../node_modules` → `frontend/node_modules` so `next` could be found from the root. Vercel's build environment does not reliably follow symlinks across directory boundaries. **Reverted.**

### Approach B — npm workspaces

**Commits:** `fix: use npm workspaces to install node_modules at repo root` → `revert: remove npm workspaces hack`

npm workspaces can hoist shared `node_modules` to the workspace root. This was attempted by adding a `workspaces` declaration to the root `package.json`. The result was a different set of resolution errors because `frontend/` and the root had overlapping but non-identical dependency declarations. Workspace hoisting caused version conflicts that were harder to resolve than the original peer dep problem. **Reverted.**

### Approach C — Move vercel.json into frontend/

**Commit:** `fix: remove 'cd frontend' from buildCommand – Vercel CWD is already frontend/`

When a `vercel.json` lives inside a subdirectory, Vercel treats that directory as the project root for build purposes — which means `installCommand` would run in `frontend/` and `next` would be findable. However this also breaks the ability to define project-level `crons` and `headers` at the repo root (Vercel only reads one `vercel.json`), and the CORS headers and cron job for `/api/cron/revalidate` must be declared at the project level.

Additionally, moving `vercel.json` into `frontend/` required removing the `cd frontend` from the `buildCommand` (since Vercel CWD would already be `frontend/`) — but this change was made prematurely and then had to be rolled back when the root `vercel.json` was restored. **Reverted.**

**Commit:** `fix: consolidate headers and crons back into root vercel.json`

Final resolution: keep `vercel.json` at root, configure explicit install/build/output paths, use `../node_modules/.bin/next build` for the binary path.

---

## 6. TypeScript Build Errors Fixed for Vercel

Vercel runs `next build`, which includes a TypeScript type check (`tsc`) as part of the build. Errors that are invisible in development (because `next dev` skips strict checking by default) surface as hard failures on Vercel.

The following issues were resolved:

### 6.1 — scripts/ directory included in tsconfig

**File:** `frontend/tsconfig.json` **Commit:** `fix: exclude scripts/ from TypeScript compilation`

The `scripts/` directory contains one-off utility scripts written in TypeScript but not intended to be part of the Next.js application build. These scripts used Node.js types and patterns incompatible with the browser/edge target of the app. The fix was to add `scripts` to the `exclude` array in `tsconfig.json`:

```
tsconfig.json
```

```
{
  "exclude": ["node_modules", "scripts"]
}
```

## 6.2 — @sanity/image-url internal type import

**Commit:** fix: remove missing @sanity/image-url internal type import

A component was importing a type from `@sanity/image-url/gen/types` — an internal path not exported by the package's public API. This path did not exist in the version installed. The fix was to remove the type import and use an inline type or `any` instead.

## 6.3 — Missing packages in package.json

**Commit:** fix: add lucide-react, chart.js, react-chartjs-2 to package.json

Three packages were imported in component code but were not declared as dependencies in `package.json`. They happened to be available locally because they were installed transitively, but Vercel's clean-install environment did not include them. The packages were added explicitly to `package.json`:

- `lucide-react` — icon library used in multiple components
- `chart.js` — charting library used in impact simulation
- `react-chartjs-2` — React wrapper for Chart.js

---

# 7. Suspense Boundary Requirement — useSearchParams

**Commit:** fix: wrap useSearchParams usages in Suspense boundaries

Next.js App Router (used by this project) requires that any component calling `useSearchParams()` be wrapped in a `<Suspense>` boundary. In development, this is a warning. On Vercel (production build), it is a hard error that halts the build:

```
Error: useSearchParams() should be wrapped in a suspense boundary at page
"/..."
```

**Pattern applied to all affected pages:**

```
// Before — fails Vercel build
export default function SomePage() {
  const searchParams = useSearchParams();
  // ...
}

// After — correct pattern
import { Suspense } from "react";

function SomePageInner() {
  const searchParams = useSearchParams();
  // ...
}

export default function SomePage() {
  return (
    <Suspense fallback={null}>
      <SomePageInner />
    </Suspense>
  );
}
```

All pages using URL search parameters were updated to follow this pattern before the Vercel build succeeded.

## 8. Security Headers — next.config.ts

Security headers are defined in `frontend/next.config.ts` and applied to all routes via the `headers()` async function. These are HTTP response headers sent by the server — not to be confused with the CORS headers in `vercel.json` (which apply at the Vercel edge layer, specifically to `/api/*` routes).

### Header inventory

| Header                    | Value                                        | Purpose                                                                       |
|---------------------------|----------------------------------------------|-------------------------------------------------------------------------------|
| X-Frame-Options           | SAMEORIGIN                                   | Prevents clickjacking — allows same-origin iframes (needed for Sanity Studio) |
| X-Content-Type-Options    | nosniff                                      | Prevents MIME-type sniffing attacks                                           |
| Referrer-Policy           | strict-origin-when-cross-origin              | Sends full URL as referrer only on same-origin requests                       |
| Permissions-Policy        | camera=(), microphone=(), geolocation=()     | Disables access to device camera, mic, and location                           |
| Strict-Transport-Security | max-age=63072000; includeSubDomains; preload | Forces HTTPS for 2 years; ignored over HTTP                                   |
| Content-Security-Policy   | See below                                    | Restricts what resources the page can load                                    |

### Content Security Policy — directive-by-directive

```
default-src 'self'
```

Fallback for all resource types not covered by a specific directive. Allows only same-origin by default.

```
script-src 'self' 'unsafe-inline' 'unsafe-eval'
          https://js.stripe.com
          https://challenges.cloudflare.com
```

- 'unsafe-inline' — required by Next.js hydration (inline `<script>` tags) and styled-components
- 'unsafe-eval' — required by Sanity Studio (`/studio` route) and certain Next.js internals
- Stripe.js — Stripe's payment form JavaScript
- Cloudflare Turnstile — bot protection challenge script

```
style-src 'self' 'unsafe-inline'
```

- 'unsafe-inline' — required by Tailwind's JIT engine and styled-components (inline `style` attributes)

```
font-src 'self' data:
```

Only self-hosted fonts and data URIs (base64-encoded fonts in CSS). No Google Fonts or external font CDNs.

```
img-src 'self' blob: data:
      https://cdn.sanity.io
      https://img.youtube.com
      https://*.tile.openstreetmap.org
```

- **blob:** — used by react-simple-maps when rendering SVG geography files
- **data:** — base64-encoded images in CSS/HTML
- Sanity CDN — all article and course images are served from Sanity's CDN
- YouTube thumbnails — editorial content embeds YouTube video thumbnails
- OpenStreetMap tile servers — the Leaflet.js map (used on the States page and hospital maps) loads tiles from **a.tile.openstreetmap.org**, **b.tile.openstreetmap.org**, **c.tile.openstreetmap.org**

```
connect-src 'self'
      https://*.supabase.co wss://*.supabase.co
      https://*.supabase.io
      https://*.api.sanity.io
      https://*.apicdn.sanity.io
      https://api.sanity.io
      https://cdn.jsdelivr.net
      https://api.stripe.com
      https://*.sentry.io
      https://o*.ingest.sentry.io
```

- Supabase REST (**https://**) and Realtime WebSocket (**wss://**) — database and auth calls
- Sanity GROQ API — content queries use **{projectId}.api.sanity.io** (two-level subdomain; **\*.sanity.io** would only match one level and would be insufficient)
- Sanity CDN GROQ — cached GROQ responses from **{projectId}.apicdn.sanity.io**
- jsDelivr CDN — US Atlas TopoJSON file used by **react-simple-maps** for state boundary rendering
- Stripe API — payment processing
- Sentry — error event ingestion

```
frame-src https://www.youtube.com
      https://open.spotify.com
      https://w.soundcloud.com
      https://js.stripe.com
      https://hooks.stripe.com
```

- YouTube — video embeds in articles and courses
- Spotify / SoundCloud — audio embeds in AudioBlock content components
- Stripe — Stripe Checkout hosted page and 3D Secure authentication iframe

```
worker-src 'self' blob:
```

Service workers and web workers. **blob:** is needed for workers created dynamically from blob URLs.

## 9. Sentry Integration

Sentry is configured in two places:

### 9.1 — next.config.ts wrapper

The entire Next.js config is wrapped with **withSentryConfig**:



```
export default withSentryConfig(nextConfig, {
  silent: true, // Suppresses Sentry CLI output during builds
  widenClientFileUpload: true, // Instruments server-side routes automatically
  sourcemaps: {
    disable: process.env.NODE_ENV !== "production", // Upload source maps only
    in CI
  },
});
```

This wrapper instruments all Next.js API routes and server components for automatic error capture. It also configures source map uploading so that Sentry error stack traces show original TypeScript line numbers rather than minified bundle positions.

### 9.2 — Environment variables required

| Variable          | Where           | Purpose                                  |
|-------------------|-----------------|------------------------------------------|
| SENTRY_DSN        | Vercel env vars | Where to send error events               |
| SENTRY_ORG        | Vercel env vars | Your Sentry organization slug            |
| SENTRY_PROJECT    | Vercel env vars | Your Sentry project slug                 |
| SENTRY_AUTH_TOKEN | Vercel env vars | Token for source map upload during build |

The `SENTRY_AUTH_TOKEN` is consumed at **build time** (not runtime). It must be set in Vercel's environment variables — not just in `.env.local`.

## 10. Environment Variables — Complete Reference

### 10.1 — Vercel (frontend) environment variables

Set these in the Vercel dashboard under **Project → Settings → Environment Variables**. Mark production/preview/development as appropriate.

#### Supabase

| Variable                                     | Exposure         | Description                                                                |
|----------------------------------------------|------------------|----------------------------------------------------------------------------|
| NEXT_PUBLIC_SUPABASE_URL                     | Public (browser) | Your Supabase project URL, e.g. <code>https://xxxxx.supabase.co</code>     |
| NEXT_PUBLIC_SUPABASE_ANON_KEY                | Public (browser) | Supabase anonymous/public key — safe to expose, governed by RLS            |
| NEXT_PUBLIC_SUPABASE_PUBLISHABLE_DEFAULT_KEY | Public (browser) | Publishable key (used in some client SDK calls)                            |
| SUPABASE_SERVICE_ROLE_KEY                    | Secret           | Service role key — bypasses RLS, server-side only, NEVER expose to browser |

#### Sanity CMS

| Variable                       | Exposure | Description                                    |
|--------------------------------|----------|------------------------------------------------|
| NEXT_PUBLIC_SANITY_PROJECT_ID  | Public   | Project ID: <code>fxz10xl7</code>              |
| NEXT_PUBLIC_SANITY_DATASET     | Public   | Dataset name: <code>production</code>          |
| NEXT_PUBLIC_SANITY_API_VERSION | Public   | API version date, e.g. <code>2024-01-01</code> |



| Variable                           | Exposure      | Description                                                                                                 |
|------------------------------------|---------------|-------------------------------------------------------------------------------------------------------------|
| <code>SANITY_API_TOKEN</code>      | <b>Secret</b> | Editor-level token for server-side content mutations and webhook verification                               |
| <code>SANITY_WEBHOOK_SECRET</code> | <b>Secret</b> | Random string used to verify Sanity webhook payloads (generated with <code>openssl rand -base64 32</code> ) |

Stripe

| Variable                                        | Exposure      | Description                                               |
|-------------------------------------------------|---------------|-----------------------------------------------------------|
| <code>NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY</code> | Public        | Stripe publishable key — used by Stripe.js in the browser |
| <code>STRIPE_SECRET_KEY</code>                  | <b>Secret</b> | Stripe secret key — server-side only                      |
| <code>STRIPE_WEBHOOK_SECRET</code>              | <b>Secret</b> | Verifies Stripe webhook payloads                          |

AI / LLM

| Variable                       | Exposure      | Description                                                                    |
|--------------------------------|---------------|--------------------------------------------------------------------------------|
| <code>ANTHROPIC_API_KEY</code> | <b>Secret</b> | Anthropic Claude API key (used by frontend Pro tier AI features)               |
| <code>OPENAI_API_KEY</code>    | <b>Secret</b> | OpenAI key (used by frontend for embeddings if Railway backend is unavailable) |

Sentry (build-time + runtime)

| Variable                       | Exposure                            | Description                                             |
|--------------------------------|-------------------------------------|---------------------------------------------------------|
| <code>SENTRY_DSN</code>        | Public (but unique to your project) | Error event ingestion endpoint                          |
| <code>SENTRY_ORG</code>        | Secret                              | Org slug for source map upload                          |
| <code>SENTRY_PROJECT</code>    | Secret                              | Project slug for source map upload                      |
| <code>SENTRY_AUTH_TOKEN</code> | <b>Secret</b>                       | Auth token for source map upload — needed at build time |

Application

| Variable                         | Exposure          | Description                                                                                                                         |
|----------------------------------|-------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>NEXT_PUBLIC_APP_URL</code> | Public            | Canonical app URL, e.g. <code>https://healthtransformationreview.com</code> — used for absolute URL generation                      |
| <code>TICKER_API_URL</code>      | Secret (optional) | If set, overrides <code>data/vitals.csv</code> as the ticker data source. Leave unset to use the CSV file.                          |
| <code>CRON_SECRET</code>         | <b>Secret</b>     | Bearer token that the cron job endpoint ( <code>/api/cron/revalidate</code> ) validates before executing. Set a long random string. |

10.2 — Railway (Python backend) environment variables

Set these in the Railway dashboard under your backend service's **Variables** tab.

| Variable                  | Description                                                           |
|---------------------------|-----------------------------------------------------------------------|
| <code>SUPABASE_URL</code> | Supabase project URL (same as <code>NEXT_PUBLIC_SUPABASE_URL</code> ) |

| Variable                  | Description                                                                                                            |
|---------------------------|------------------------------------------------------------------------------------------------------------------------|
| SUPABASE_SERVICE_ROLE_KEY | Service role key (same as frontend secret)                                                                             |
| SUPABASE_DB_URL           | Direct PostgreSQL connection string: <code>postgresql://postgres:[pw]@db.[project-id].supabase.co:5432/postgres</code> |
| SANITY_PROJECT_ID         | Sanity project ID: <code>fxz10xl7</code>                                                                               |
| SANITY_DATASET            | Dataset: <code>production</code>                                                                                       |
| SANITY_API_TOKEN          | Editor-level Sanity token (same as frontend)                                                                           |
| SANITY_WEBHOOK_SECRET     | Webhook verification secret (same as frontend)                                                                         |
| GROQ_API_KEY              | Groq API key for LLM inference (fast llama/mixtral models)                                                             |
| OPENAI_API_KEY            | OpenAI API key for embeddings ( <code>text-embedding-3-small</code> ) and TTS ( <code>tts-1</code> )                   |
| ANTHROPIC_API_KEY         | Anthropic API key for Claude-powered AI Analyst responses                                                              |
| PORT                      | Railway sets this automatically — do not override                                                                      |

## 11. Cron Job — Nightly Revalidation

Declared in `vercel.json`:

```
"crons": [  
  {  
    "path": "/api/cron/revalidate",  
    "schedule": "0 0 * * *"  
  }  
]
```

This calls `/api/cron/revalidate` every day at midnight UTC. The endpoint triggers Next.js ISR (Incremental Static Regeneration) cache revalidation for pages with time-sensitive content (ticker data, article lists, etc.).

**Security:** The endpoint must validate the `CRON_SECRET` environment variable. Vercel sends the cron request with an `Authorization: Bearer <secret>` header. The route handler should reject any call that does not present the correct bearer token. Set `CRON_SECRET` to a long random string in Vercel's environment variables.

```
// Example guard in app/api/cron/revalidate/route.ts  
export async function GET(request: Request) {  
  const auth = request.headers.get("authorization");  
  if (auth !== `Bearer ${process.env.CRON_SECRET}`) {  
    return new Response("Unauthorized", { status: 401 });  
  }  
  // ... revalidation logic  
}
```

Cron jobs require Vercel **Pro** plan or higher. On the free Hobby plan the cron declaration is ignored.

## 12. CORS Headers

Declared in `vercel.json`:

```
"headers": [  
  {  
    "source": "/api/(.*)",
```

```
    "headers": [
      { "key": "Access-Control-Allow-Methods", "value": "GET, POST, OPTIONS" },
      { "key": "Access-Control-Allow-Headers", "value": "Content-Type,
Authorization" }
    ]
  }
]
```

These headers are injected by the Vercel edge layer on all `/api/*` responses. They allow the Railway backend (and any other authorized client) to call the Next.js API routes cross-origin.

**Note:** `Access-Control-Allow-Origin` is intentionally absent here. When it needs to be set (e.g., restricting to the Railway domain), it should be added to this header list and scoped appropriately. Currently, API routes that require stricter origin control handle `Access-Control-Allow-Origin` within the route handler itself.

### 13. Relationship to Existing deployment-guide.md

`docs/deployment-guide.md` (v4.2.0) is the first-deployment walkthrough. It covers:

- Account creation for all 8 services
- Supabase project setup and all 19 migration files
- Sanity project and webhook setup
- Stripe payment configuration
- Railway backend deployment
- Vercel — but only the happy-path: connecting the repo and setting env vars

It does **not** cover:

- The monorepo structure and why it requires explicit `vercel.json` overrides
- The `@sentry/nextjs` peer dependency conflict and the `legacy-peer-deps` fix
- The `next` binary path issue (`../node_modules/.bin/next build`)
- The failed approaches (symlinks, workspaces, frontend-level `vercel.json`)
- TypeScript build errors that only surface in Vercel's production build
- The `useSearchParams` Suspense boundary requirement
- The full CSP header rationale (OpenStreetMap, Sanity two-level subdomain, etc.)
- The cron job security pattern

This amendment (v4.9.0) fills those gaps. When onboarding a new developer, both documents should be read — `deployment-guide.md` for service setup, this amendment for the configuration decisions and known friction points.

### 14. Deployment Checklist

Use this when deploying to a new Vercel environment or debugging a failed build.

#### Pre-deployment

- ☐ `vercel.json` is at the **repo root** (not inside `frontend/`)
- ☐ `.npmrc` at repo root contains `legacy-peer-deps=true`
- ☐ Root `package.json` lists all packages (including `lucide-react`, `chart.js`, `react-chartjs-2`)
- ☐ `frontend/tsconfig.json` excludes `scripts` directory

#### Vercel project settings

- ☐ **Root Directory** in Vercel project settings is set to `/` (repo root) — not `frontend/`
- ☐ All environment variables listed in §10.1 are set for the appropriate environments
- ☐ `SENTRY_AUTH_TOKEN` is set (required at build time, not just runtime)
- ☐ `CRON_SECRET` is set (required for cron job endpoint security)

#### Build verification

- ☐ Build logs show `npm install` running at repo root
- ☐ Build logs show `cd frontend && ../node_modules/.bin/next build`
- ☐ No TypeScript errors in build log
- ☐ No `useSearchParams` Suspense boundary errors in build log
- ☐ Output directory `frontend/.next` is found by Vercel

## Post-deployment

- ☐ Supabase **Site URL** updated to production domain (Authentication → URL Configuration)
- ☐ Sanity webhook configured to call `https://your-domain.com/api/webhooks/sanity`
- ☐ Stripe webhook configured to call `https://your-domain.com/api/webhooks/stripe`
- ☐ Sentry release created and source maps uploaded (should happen automatically via `withSentryConfig`)
- ☐ `/api/cron/revalidate` returns 200 when called with correct `Authorization: Bearer <CRON_SECRET>` header